



Записка_до_курсової_роботи_Астененко.docx

04/30/2026 5:04 AM

1%

Plagiarised

99%

Unique

Quality Metrics

Readability: **55% (Medium)**

Verbosity: **15% (Low)**

Repetition: **Low**

Informativeness: **High**

Clichés detected: **0**

Writing Analysis

Sentiment: **Neutral**

Formality: **Formal**

Intent: **Inform**

Style: **Technical Documentation / Academic
Project Report**

Topic: **Computer Science / Software
Engineering**

МІНІСТЕРСТВО ОСВІТИ И НАУКИ УКРАЇНИ

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені І.І. МЕЧНИКОВА

Факультет математики, фізики та інформаційних технологій

Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЕКТ

з дисципліни

«Організація баз даних»

на тему:

«Інформаційна система міського транспорту»

студента ІІІ курсу Станіслав Астененко

групи І_____

спеціальності 123 «Комп'ютерна інженерія»

Керівник: **ст. викл. Розновець О.І.**_____

Комісія:

(ПІБ)	(Підпис)
(ПІБ)	(Підпис)
(ПІБ)	(Підпис)

Одеса – 2025

АНОТАЦІЯ

Курсовий проєкт присвячено проєктуванню та реалізації інформаційної системи для управління міським громадським транспортом. Метою курсового проєкту є проєктування та реалізація інформаційної системи для управління міським транспортом, що дозволяє адміністраторам, водіям і користувачам взаємодіяти в межах єдиного цифрового середовища.

Система орієнтована на вирішення проблем відсутності централізованого обліку транспорту, складності контролю маршрутів і неефективної комунікації між учасниками транспортного процесу. У межах системи передбачено розмежування ролей користувачів: адміністраторів, водіїв і пасажирів, кожен з яких має відповідні функціональні можливості. Адміністратори здійснюють керування транспортними засобами та маршрутами, водії отримують доступ до закріплених маршрутів і транспорту, а пасажирів мають змогу переглядати інформацію про маршрути й зупинки.

Реалізація системи базується на сучасних вебтехнологіях: NestJS для серверної частини, Next.js для клієнтського інтерфейсу, PostgreSQL для зберігання даних і TypeORM для взаємодії з базою даних. Архітектура застосунку побудована за багаторівневим принципом, що забезпечує чітке розділення логіки, зручність масштабування та підтримки.

У проєкті реалізовано механізми автентифікації та авторизації користувачів, валідацію даних і контроль доступу відповідно до ролей, що гарантує цілісність і безпеку інформації. Результатом роботи є функціональна інформаційна система управління міським транспортом, яка підвищує ефективність роботи транспортних служб і забезпечує користувачам доступ до актуальної інформації.

ЗМІСТ

ВСТУП 4

1 ПОСТАНОВКА ЗАДАЧІ 6

2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ 10

2.1 Архітектура ІС 10

2.2 Шаблон проектування	11
3 ІНФОРМАЦІЙНЕ МОДЕЛЮВАННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	13
3.1 Структура інформаційної системи	13
3.2 Формалізація зв'язків між сутностями	16
4 ПРОГРАМНА МОДЕЛЬ ЗАСТОСУНКА	18
5 ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ СТВОРЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ	21
6 СТВОРЕННЯ БАЗИ ДАНИХ	23
7 ЗАПИТИ ДО БАЗИ ДАНИХ ДЛЯ РОЗВ'ЯЗАННЯ ПОСТАВЛЕНИХ ЗАДАЧ	25
8 ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ	31
8.1 Підключення до бази даних	31
8.2 Призначення транспортного засобу	32
8.3 Динамічне формування HTML-сторінки (фрагмент шаблону)	33
9 БЕЗПЕКА ІНФОРМАЦІЙНОЇ СИСТЕМИ	34
9.1. Ролі користувачів та їхні привілеї	34
9.2 Механізм зберігання паролів	36
9.3 Алгоритм автентифікації та визначення ролі користувача	37
9.4 Додаткові механізми захисту	38
10 ІНСТРУКЦІЯ КОРИСТУВАЧА	40
10.1 Інтерфейс пасажирів	40
10.2 Інтерфейс водія	43
10.3 Інтерфейс адміністратора.	44
ВИСНОВКИ	47
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	48
ДОДАТОК А Запити на створення таблиць бази даних	49
ДОДАТОК Б Запити на створення тригерів та функції для вирішення задач ІС	52
ДОДАТОК В Основні модулі NestJs	54

ВСТУП

У сучасних умовах розвитку міської інфраструктури важливе значення має ефективна організація системи громадського транспорту. Щодня тисячі пасажирів користуються автобусами, трамваями та іншими видами транспорту, що потребує точного планування маршрутів, контролю руху та зручної комунікації між адміністраторами, водіями та користувачами. Незважаючи на наявність окремих сервісів для відстеження транспорту чи планування маршрутів, більшість із них не забезпечують повноцінного управління всіма процесами – від адміністрування транспортних засобів до контролю роботи персоналу.

Крім цього, адміністрації міських перевізників стикаються з проблемами відсутності централізованої системи обліку транспорту, неефективною комунікацією між працівниками та пасажирами, а також складністю у зборі й аналізі даних щодо маршрутів і зупинок. Водії, у свою чергу, потребують зручного інструменту для перегляду закріплених транспортних засобів, маршрутів та графіків. Пасажири очікують на прозору систему, де можна отримати достовірну інформацію про рух транспорту, маршрути та зупинки у зручній формі.

Таким чином, постає потреба у створенні універсальної інформаційної системи міського транспорту, яка забезпечить автоматизацію управління транспортними процесами, взаємодію між різними категоріями користувачів і централізоване зберігання даних. Запропонована система має підвищити ефективність роботи транспортних служб, зменшити адміністративне навантаження та забезпечити комфорт пасажирів завдяки доступу до актуальної інформації.

Метою курсового проєкту є проєктування та реалізація інформаційної системи для управління міським транспортом, що дозволяє адміністраторам, водіям і користувачам взаємодіяти в межах єдиного цифрового середовища. Система має забезпечувати можливість перегляду маршрутів, обліку транспортних засобів, керування ролями користувачів, а також подання заявок на роботу водієм.

Для досягнення поставленої мети необхідно вирішити такі основні завдання:

Провести аналіз предметної області міського транспорту та визначити ключові поняття, бізнес-процеси й вимоги до системи.

Визначити категорії користувачів (адміністратор, водій, пасажир) і описати їхні потреби та функціональні можливості.

Обрати архітектурний підхід і відповідний шаблон проєктування для реалізації багаторівневої вебсистеми.

Спроєктувати структуру бази даних, що забезпечить зв'язки між користувачами, транспортом, маршрутами та зупинками.

Вибрати сучасні технології розробки — NestJS, Next.js, PostgreSQL і TypeORM — для реалізації серверної та клієнтської частини системи.

Реалізувати базу даних і користувацький інтерфейс, який забезпечить зручну роботу з даними відповідно до ролі користувача.

Забезпечити цілісність і безпеку даних шляхом аутентифікації, авторизації та валідації введеної інформації.

Реалізувати механізми контролю доступу, захисту від несанкціонованих дій та централізоване логування подій.

1 ПОСТАНОВКА ЗАДАЧІ

Система, що розробляється, призначена для автоматизації процесів управління міським транспортом, забезпечення ефективної взаємодії між адміністраторами, водіями та користувачами, а також централізованого зберігання та обробки даних про маршрути, транспорт і зупинки. Інформаційна система надає можливість адмініструвати транспортну інфраструктуру міста, контролювати роботу перевізників і забезпечувати громадян актуальною інформацією про рух транспорту.

Основними користувачами системи є:

Адміністратор — здійснює контроль за транспортом, управління користувачами, призначення маршрутів і схвалення заявок користувачів на отримання роботи водія та веде аналіз найчастіше використовуваних маршрутів.

Водій — переглядає закріплений транспортний засіб і маршрути, оновлює статус рейсів.

Користувач — переглядає доступні маршрути, транспортні засоби, зупинки.

Задачі, які вирішуються інформаційною системою:

Реєстрація (ім'я, пошта та пароль користувача) та автентифікація користувачів із розмежуванням доступу відповідно до ролей (адміністратор, водій, користувач).

Облік транспортних засобів та їхніх характеристик (тип, номер, місткість, водій).

Перегляд статистики популярних маршрутів за кількістю транспортів на маршруті.

Формування та редагування маршрутів із можливістю додавання зупинок.

Подання заявок користувачами на отримання ролі водія та їх схвалення або відхилення адміністратором.

Перегляд маршрутів і транспортних засобів користувачами.

Контроль і оновлення даних про водіїв, транспорт і маршрути адміністраторами.

Додавання нових маршрутів.

Додавання зупинок до маршруту.

Аналіз найчастіше використовуваних маршрутів за кількістю транспорту.

Подання заявки на роботу водієм.

Список задач (з описом вхідних та вихідних даних) користувачів ІС «Міський транспорт» наведений в табл. 1.1 – 1.3.

Таблиця 1.1 – Список задач користувача адміністратор ІС «Міський транспорт»

Номер

Задачі

Вхідні дані

Вихідні дані

A1

Перегляд усіх транспортних засобів

Список транспорту (номер, тип, водій, місткість)

A2

Додавання нового транспорту

Номер, тип, місткість, стан транспорту

Новий транспорт з унікальним ID

A3

Призначенн маршруту

транспорту

ID транспорт,

ID маршруту

Оновлений запис транспорту з призначеним маршрутом

A4

Розгляд заявок на роботу водіїв

ID заявки, статус

(approved, rejected)

Заявка з оновленим статусом

Продовження таблиці 1.1

Номер

Задачі

Вхідні дані

Вихідні дані

A5

Перегляд користувачів системи

Список користувачів (ім'я, прізвище, роль, пошта)

A6

Зміна ролі користувача

ID користувача,

нова роль

Оновлена роль користувача

A7

Видалення транспорту

ID запису

Підтвердження видалення

A8

Видалення маршруту

ID запису

Підтвердження видалення

A9

Додавання користувача

(користувач або водій в залежності від вибраної ролі)

Ім'я, прізвище, вік, пошта, пароль, роль

Новий користувач (користувач або водій) з унікальним ID

A10

Додавання маршруту

Зупинки, назва маршруту, дистанція маршруту в км, назви зупинок

Новий маршрут з унікальним ID

A11

Перегляд статистики маршрутів

Назва маршруту, кількість транспорту

Таблиця 1.2 - Список задач користувача водій IC «Міський транспорт»

Номер

Задача

Вхідні дані

Вихідні дані

V1

Перегляд власного транспорту

ID користувача

Дані про транспорт (номер, тип,

кількість міст, стан транспорту)

V2

Перегляд маршруту

ID транспорту

Список зупинок і кінцевих пунктів (порядок зупинок на маршруті визначаються унікальним порядковим номером)

V3

Перегляд особистої інформації

ID користувача

Дані профілю (ПІБ, роль)

Таблиця 1.3 – Список задач користувача пасажир(користувач) ІС «Міський транспорт»

Номер

Задача

Вхідні дані

Вихідні дані

U1

Перегляд конкретного маршруту

ID маршруту

Деталі: початкова/кінцева точка, зупинки (порядок зупинок на маршруті визначаються унікальним порядковим номером)

U2

Подання заявки на роботу водієм

ID користувача, бажана роль

Нова заявка з унікальним ID

U3

Перегляд статусу заявки

ID користувача

Поточний статус

заявки

2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

2.1 Архітектура ІС

За основу розроблюваної інформаційної системи «Міський транспорт» обрано тришарову клієнт-серверну архітектуру (рис. 2.1), яка забезпечує логічне розділення системи на рівні представлення, бізнес-логіки та роботи з даними. Такий підхід підвищує масштабованість, гнучкість та безпеку системи, дозволяючи незалежно оновлювати окремі її компоненти.

Рисунок 2.1 – Трирівнева архітектура клієнт-сервер

Шар представлення відповідає за взаємодію користувача із системою. Через цей рівень відбувається відображення інформації та введення даних користувачем. Система передбачає роботу з кількома категоріями користувачів, зокрема пасажирями, водіями та адміністраторами. Основна функція цього шару полягає у відображенні даних, отриманих із серверної частини, та передачі дій користувача для подальшої обробки. Користувачі можуть переглядати інформацію про маршрути, зупинки та транспортні засоби, а також подавати заявки на роботу водієм. Інтерфейс системи побудований з урахуванням сучасних принципів зручності використання, що забезпечує зрозумілу навігацію та адаптивність.

Шар бізнес-логіки відповідає за обробку запитів користувачів і реалізацію основних функцій системи. На цьому рівні виконуються перевірка прав доступу, авторизація користувачів, валідація введених даних та формування відповідей для клієнтської частини.

Шар бізнес-логіки містить основні процеси предметної області, серед яких:

Керування користувачами та ролями (адміністратор, водій, користувач).

Облік транспортних засобів та маршрутів.

Обробка заявок на роботу водієм.

Контроль даних про зупинки – збереження назви зупинки маршруту та її порядкового номера з забезпечення унікальної послідовності в межах маршруту.

Надання структурованих даних для відображення на клієнті (список маршрутів, список зупинок на кожному маршруті у правильному порядку, інформацію про транспортні засоби, інформацію про користувачів).

Такий підхід забезпечує централізовану обробку даних і логіки системи, що спрощує підтримку та подальший розвиток інформаційної системи.

Рівень даних призначений для зберігання та організації інформації, що використовується в інформаційній системі. На цьому рівні передбачається формування структури даних та встановлення зв'язків між окремими сутностями предметної області. У базі даних планується зберігати відомості про користувачів, транспортні засоби, маршрути, зупинки та заявки на роботу. Така організація даних дозволить забезпечити їх цілісність, впорядкованість та ефективний доступ до інформації під час роботи системи.

2.2 Шаблон проектування

Для організації логіки та структури застосунку інформаційної системи «Міський транспорт» обрано шаблон проектування MVC (Model-View-Controller), який забезпечує розділення відповідальностей між компонентами системи, покращує масштабованість і полегшує супровід проекту.

У межах реалізації застосунку кожен із компонентів MVC виконує свою роль у взаємодії користувача з системою.

На рисунку 2.2 зображено розподіл відповідальностей між трьома основними компонентами шаблону MVC. Користувач взаємодіє з інтерфейсом, створеним засобами View, який у відповідь надсилає запит до Controller. Контролер опрацьовує логіку запиту, звертається до відповідної моделі Model, отримує або змінює дані в базі, після чого повертає результат назад до View для відображення.

Рисунок 2.2 – Шаблон MVC

Використання шаблону MVC забезпечує відповідність структурі трирівневої архітектури системи: шар представлення відповідає за взаємодію з користувачем, контролер – за бізнес-логіку, а модель – за роботу з даними.

Серед основних переваг підходу варто виділити:

Чітке розділення відповідальностей між компонентами.

Легке тестування та масштабування застосунку.

Зручність у супроводі та розширенні функціоналу.

Зменшення зв'язності між клієнтською, серверною та базовою частинами.

У порівнянні з іншими архітектурними підходами (MVP, MVVM), модель MVC є оптимальною для вебсистем, оскільки вона поєднує структурованість, логічну ізоляцію модулів і простоту подальшої підтримки.

3 ІНФОРМАЦІЙНЕ МОДЕЛЮВАННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

3.1 Структура інформаційної системи

Опис сутностей предметної області «Міський транспорт» наведено в таблиці 3.1. Кожна сутність відображає реальний об'єкт або процес, що існує у межах системи: користувачів, транспортні засоби, маршрути, зупинки, ролі та заявки на роботу. Для кожної сутності визначено її атрибути, функціональне призначення та обмеження цілісності – типи ключів, обов'язковість заповнення та допустимі значення.

Таке структурування забезпечує логічну узгодженість даних, спрощує побудову запитів і гарантує цілісність інформації. Наприклад, використання зовнішніх ключів забезпечує зв'язок між транспортом і водієм, а між маршрутами та зупинками – ієрархічну залежність. Додаткові обмеження, як-от унікальність електронної пошти або статус заявки, забезпечують коректність і достовірність даних у системі.

Таблиця 3.1 – Опис сутностей предметної області «Міський транспорт»

Ім'я атрибута

Призначення атрибута

Обмеження

Roles (Ролі)

role_id

ідентифікатор ролі

первинний ключ

role_name

назва ролі

унікальне, не порожнє

Users(Всі користувачі)

user_id

Ідентифікатор користувача

Первинний ключ

name

Ім'я користувача

Не порожнє

email

Пошта користувача

Не порожнє (назва_користувача@домен.зона)

Продовження таблиці 3.1

password

Пароль користувача

Не порожнє

age

Вік користувача

Не порожнє (від 14 – до 120)

role_id

Ідентифікатор ролі користувача

Зовнішній ключ, не порожнє, посилається на таблицю roles на поле role_id

Transport (Транспорт)

transport_id

Ідентифікатор транспорту

Первинний ключ

number

Номер транспорту

Не порожнє (2 літери + 4 цифри + 2 літери)

type

Тип транспорту

Значення з множини (автобус, тролейбус, трамвай)

capacity

Пасажиромісткість

Ціле число більше 0 та менше 35

driver_id

Ідентифікатор водія

Зовнішній ключ, посилається на таблицю users на поле user_id

Transport_condition

Стан транспорту

Не порожнє

route_id

Ідентифікатор маршруту

Зовнішній ключ, посилається на таблицю routes на поле route_id

Routes (Маршрути)

route_id

Ідентифікатор маршруту

Первинний ключ

name

Назва маршруту

Не порожнє

distance

Дистанція маршруту

Не порожнє більше або дорівнює 1км та менше або дорівнює 30км

Продовження таблиці 3.1

Stops (Зупинки)

stop_id

Ідентифікатор зупинки

Первинний ключ

stop_name

Назва зупинки

Не порожнє

Route_stops (проміжна таблиця між таблицями stops та route)

id

Ідентифікатор запису

Первинний ключ

stopId

Ідентифікатор зупинки

Зовнішній ключ, посилається на таблицю stops на поле stop_id

route_id

Ідентифікатор маршруту

Зовнішній ключ, посилається на таблицю routes на поле route_id

stop_order

Порядковий номер зупинки в межах маршруту

Ціле число більше 1 та менше 11

Applications (Заявки)

applications_id

Ідентифікатор заявки

Первинний ключ

user_id

Ідентифікатор користувача

Зовнішній ключ, посилається на таблицю users на поле user_id

role_id

Ідентифікатор ролі

Зовнішній ключ, посилається на таблицю roles на поле role_id

status

Статус заявки

Значення з множини (pending, approved, rejected)

created_at

Дата створення заявки

Заповнення автоматично поточною датою та часом на момент створення заявки

Продовження таблиці 3.1

decision_at

Дата прийняття рішення

Заповнення автоматично поточною датою та часом на момент прийняття заявки

3.2 Формалізація зв'язків між сутностями

У створеній інформаційній системі «Міський транспорт» реалізовано два основні типи зв'язків між сутностями: один-до-багатьох (1:N) та багато-до-багатьох (M:N). Такі зв'язки відображають реальні взаємозалежності між об'єктами транспортної мережі – користувачами, транспортом, маршрутами, зупинками та заявками.

Для формалізації зв'язків типу 1:N (один-до-багатьох) у системі кожна сутність представлена окремою таблицею, а первинний ключ відношення, що відповідає сутності «один», додається як зовнішній ключ у таблицю сутності «багато». Наприклад, між сутностями Transport (транспорт) та Routes (маршрути) існує зв'язок типу «один-до-багатьох», оскільки один транспортний засіб може обслуговувати декілька маршрутів, але кожен маршрут належить лише одному транспортному засобу. Для реалізації цього зв'язку до таблиці Routes додано поле `transport_id`, яке виступає зовнішнім ключем, що посилається на Transport(`transport_id`).

Подібним чином реалізовано інші зв'язки типу 1:N у системі:

Roles ↔ Users — кожна роль може належати багатьом користувачам, але кожен користувач має лише одну роль.

Users ↔ Transport — один водій може керувати кількома транспортними засобами.

Routes ↔ Transport — один маршрут має багато транспорту.

Routes ↔ Stops — один маршрут має багато зупинок.

Users ↔ Applications — один користувач може подати кілька заявок, але кожна заявка належить лише одному користувачу.

Для реалізації зв'язків типу M:N (багато-до-багатьох) у системі використовуються проміжні таблиці (асоціативні сутності), які містять пари зовнішніх ключів, що посилаються на первинні ключі пов'язаних таблиць. У разі, якщо таке відношення не має власного ідентифікатора, первинним ключем виступає комбінована множина з двох зовнішніх ключів.

Як приклад зв'язку M:N — між таблицями Routes та Stops, коли одна зупинка може належати кільком маршрутам (наприклад, перетин двох ліній транспорту).

Отже, у системі «Міський транспорт» реалізовано як ієрархічні (1:N), так і асоціативні (M:N) зв'язки між сутностями, що забезпечують логічну цілісність і повноту моделі даних. Візуальне представлення зв'язків між сутностями у вигляді ER-діаграми наведено у додатку Д (рисунок Д.1). Для побудови діаграми використано програмне середовище Draw.io, яке дозволяє графічно відобразити зв'язки між таблицями та забезпечує наочне розуміння структури бази даних.

4 ПРОГРАМНА МОДЕЛЬ ЗАСТОСУНКА

Даний розділ присвячений побудові програмної моделі інформаційної системи міського транспорту, що реалізує автоматизацію процесів керування маршрутами, транспортом, водіями та користувачами. Модель відображає логічну структуру системи, декомпозицію її компонентів на підсистеми та організацію взаємодії між ними через REST API.

На рис. 4.1 зображено ієрархію форм підсистеми користувача Водій. Дана підсистема забезпечує перегляд закріпленого транспорту, актуальних маршрутів та зупинок.

Рисунок 4.1 – Ієрархія форм підсистеми користувача Водій

На рис. 4.2 представлено ієрархію форм для підсистеми користувача Адміністратор. Адміністратор має розширені повноваження: керування користувачами (додавання, видалення, зміна ролей), створення й редагування маршрутів, закріплення транспорту за водіями, перегляд заявок на роботу та прийняття рішень щодо їх затвердження або відхилення.

Рисунок 4.2 – Ієрархія форм підсистеми користувача Адміністратор

На рис. 4.3 наведена ієрархія форм для підсистеми користувача Пасажир. Ця підсистема забезпечує зручний перегляд доступного транспорту, маршрутів та зупинок, а також отримання актуальної інформації щодо руху міського транспорту

Рисунок 4.3 – Ієрархія форм підсистеми користувача Пасажир

5 ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ СТВОРЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

Інформаційна система міського транспорту розроблена у форматі веб-застосунку, що забезпечує зручний доступ до функціоналу через будь-який сучасний веб-браузер. Такий підхід дозволяє користувачам (пасажирам, водіям і адміністраторам) взаємодіяти із системою у реальному часі, незалежно від типу пристрою.

Для реалізації системи обрано сучасний стек технологій, що поєднує стабільність, безпеку та масштабованість:

PostgreSQL – система керування базами даних (СКБД), яка використовується для збереження інформації про користувачів, маршрути, транспорт, зупинки та заявки. PostgreSQL забезпечує високу надійність, підтримку транзакцій і засоби забезпечення цілісності даних, що особливо важливо для багатокористувацьких систем.

NestJS – серверний фреймворк на мові TypeScript, який використовується для реалізації бізнес-логіки, обробки запитів і роботи з базою даних через ORM. NestJS базується на архітектурі модулів, що дозволяє створювати гнучку, розширювану структуру коду. Серверна частина реалізує REST API, що обробляє запити від клієнтської частини, здійснює перевірку прав доступу та забезпечує взаємодію з базою даних. Завдяки використанню Guards, Interceptors і Decorators, досягнуто чітке розмежування ролей користувачів (адміністратор, водій, пасажир).

Next.js – сучасний фреймворк для створення інтерфейсів користувача. Його застосування дозволило реалізувати серверний рендеринг (SSR), що забезпечує високу швидкість завантаження сторінок і поліпшує користувацький досвід. Інтерфейс розроблено з використанням React-компонентів, що забезпечує адаптивність, динамічність і зручність використання для користувачів різних категорій.

TypeORM – ORM-бібліотека, яка використовується для зв'язку між кодом застосунку та базою даних PostgreSQL. Вона дозволяє розробнику працювати з таблицями через класи та об'єкти, а не безпосередньо через SQL-запити. Це забезпечує безпечну взаємодію із даними, захищає від SQL-ін'єкцій і полегшує модифікацію структури бази без втрати узгодженості. Завдяки підтримці

зв'язків між сутностями (OneToMany, ManyToOne, ManyToMany) реалізовано логічні відношення між користувачами, транспортом і маршрутами.

Tailwind CSS і Material UI – бібліотеки для оформлення інтерфейсу. Вони забезпечують візуальну узгодженість, адаптивність до різних пристроїв і можливість швидкої розробки сучасного дизайну без перевантаження сторінок.

Visual Studio Code – інтегрована середовище розробки (IDE), яка застосовувалася для написання, тестування та налагодження коду. Використання розширень для NestJS, TypeScript і PostgreSQL дозволило оптимізувати процес розробки.

Завдяки поєднанню вищезазначених технологій створено гнучку, надійну та безпечну систему, яка легко масштабується та підтримує розмежування ролей користувачів. Архітектура побудована з урахуванням принципів модульності, розширюваності та повторного використання компонентів, що забезпечує простоту подальшої підтримки та розвитку проєкту.

6 СТВОРЕННЯ БАЗИ ДАНИХ

У межах системи реалізовано таблиці, представлення, послідовності, зовнішні ключі та тригери для підтримання цілісності та узгодженості даних. Повний SQL-код створення таблиць наведено у додатку А.

Розглянемо запит на створення таблиці Users (лістинг 6.1):

```
CREATE TABLE users (  
  
  userId SERIAL PRIMARY KEY,  
  
  name VARCHAR,  
  
  surname VARCHAR,  
  
  age INTEGER,  
  
  email VARCHAR NOT NULL,  
  
  password VARCHAR NOT NULL,  
  
  "roleId" INTEGER,  
  
  CONSTRAINT pk_users PRIMARY KEY ("userId"),
```

CONSTRAINT uq_users_email

UNIQUE (email),

CONSTRAINT fk_users_role

FOREIGN KEY ("roleId")

REFERENCES public.roles ("roleId")

ON DELETE SET NULL,

CONSTRAINT chk_users_age

CHECK (age IS NULL OR age BETWEEN 14 AND 120),

CONSTRAINT chk_users_email_format

CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$')

);

Лістинг 6.1 – Запит на створення таблиці Users

Основні поля таблиці мають наступне призначення:

UserID – первинний ключ, який автоматично генерується за допомогою типу SERIAL, забезпечуючи унікальність записів.

Name – ім'я користувача.

Surname – фамілія користувача.

Email – електронна адреса користувача, яка проходить перевірку формату за допомогою регулярного виразу та має унікальне значення.

Password – збережений у хешованому вигляді пароль користувача.

RoleId – зовнішній ключ до таблиці Roles, що визначає рівень доступу користувача (пасажир, водій, адміністратор).

Таким чином, структура таблиці Users забезпечує зберігання базових даних про користувачів, контроль унікальності та відповідність введених даних вимогам системи.

7 ЗАПИТИ ДО БАЗИ ДАНИХ ДЛЯ РОЗВ'ЯЗАННЯ ПОСТАВЛЕНИХ ЗАДАЧ

У цьому розділі наведено основні SQL-запити, які реалізовані в межах розробленої інформаційної системи міського транспорту. Повний текст SQL-запитів і тригерів подано у додатку Б.

У наведених SQL-запитах використовуються параметризовані значення, позначені у вигляді :parameter_name (наприклад, :route_id, :user_id). Такі параметри не є фіксованими значеннями, а передаються із програмної частини застосунку під час виконання запиту.

Цей підхід дозволяє:

динамічно підставляти необхідні значення залежно від дій користувача;

підвищити безпеку системи за рахунок захисту від SQL-ін'єкцій;

забезпечити повторне використання SQL-запитів у різних сценаріях роботи системи.

Запити для звичайних користувачів

Користувачі системи мають доступ до перегляду інформації про маршрути, транспорт і зупинки, а також можуть подати заявку на отримання ролі водія.

Перегляд усіх маршрутів (задача U1)(лістинг 7.1):

```
SELECT * FROM routes
```

```
ORDER BY route_id;
```

Лістинг 7.1 – Перегляд усіх маршрутів

Перегляд транспорту на певному маршруті (задача U1)(лістинг 7.2):

```
SELECT t.transport_id, t.number, t.type, t.capacity
```

```
FROM transport t
```

```
JOIN routes r ON t.route_id = r.route_id
```

```
WHERE r.route_id = :route_id; // підставити значення //ідентифікатора маршруту, яке буде передане програмою
```

Лістинг 7.2 – Перегляд транспорту на певному маршруті

Перегляд зупинок маршруту (задача U1)(лістинг 7.3):

```
SELECT stop_name, stop_order
```

FROM stops

WHERE route_id = :route_id

ORDER BY stop_order;

Лістинг 7.3 – Перегляд зупинок маршруту

Подання заявки на роботу водієм (задача U2, 11)(лістинг 7.4):

```
INSERT INTO applications (user_id, desired_role_id, status)
```

```
VALUES (:user_id, 2, 'pending');
```

Лістинг 7.4 – Подання заявки на роботу водієм

Цей запит дозволяє користувачеві подати заявку на зміну ролі з «користувача» на «водія». Після подання заявки запис автоматично потрапляє до таблиці applications зі статусом pending («на розгляді»).

Запити для водіїв

Після схвалення заявки адміністратором користувач отримує роль водія та доступ до функціоналу управління транспортом і маршрутами.

Перегляд призначеного транспорту (задача V1)(лістинг 7.5):

```
SELECT * FROM transport
```

```
WHERE driver_id = :user_id;
```

Лістинг 7.5 – Перегляд призначеного транспорту

Перегляд маршруту свого транспорту (V2)(лістинг 7.6):

```
SELECT r.route_id, r.start_point, r.end_point
```

```
FROM routes r
```

```
JOIN transport t ON r.transport_id = t.transport_id
```

```
WHERE t.driver_id = :user_id;
```

Лістинг 7.6 – Перегляд маршруту свого транспорту

Перегляд списку всіх зупинок (V2)(лістинг 7.7):

```
SELECT s.stop_name, s.stop_orderFROM stops sJOIN routes r ON s.route_id = r.route_idJOIN transport t ON t.route_id = r.route_idWHERE t.driver_id = :user_idORDER BY s.stop_order;
```

Лістинг 7.7 – Перегляд списку всіх маршрутів

Оновлення інформації про технічний стан транспорту (задача A2, V1) (лістинг 7.8):

```
UPDATE transport
```

```
SET transport_condition = :status
```

```
WHERE transport_id = :transport_id;
```

Лістинг 7.8 – Оновлення інформації про технічний стан транспорту

Таким чином, водій може повідомляти про технічні несправності чи завершення рейсу, змінюючи статус транспорту.

Запити для адміністратора

Адміністратор має повний доступ до системи — може керувати користувачами, транспортом, маршрутами, заявками та контролювати актуальність даних.

Перегляд усіх користувачів (задача A5) (лістинг 7.9):

```
SELECT u.user_id, u.name, u.email, r.role_name, u.is_active
```

```
FROM users u
```

```
JOIN roles r ON u.role_id = r.role_id;
```

Лістинг 7.9 – Перегляд усіх користувачів

Зміна ролі користувача (задача A6) (наприклад, після схвалення заявки) (лістинг 7.10):

```
UPDATE users
```

```
SET role_id = :new_role_id
```

```
WHERE user_id = :user_id;
```

Лістинг 7.10 – Зміна ролі користувача

Перегляд усіх заявок (задача A4) (лістинг 7.11):

```
SELECT a.app_id, u.name, r.role_name AS desired_role, a.status
```

```
FROM applications a
```

```
JOIN users u ON a.user_id = u.user_id
```

```
JOIN roles r ON a.desired_role_id = r.role_id;
```

Лістинг 7.11 – Перегляд усіх заявок

Оновлення статусу заявки (задача A4) (лістинг 7.12):

UPDATE applications

SET status = 'approved'

WHERE app_id = :app_id;

Лістинг 7.12 – Оновлення статусу заявки

Додавання нового транспорту (задача A2)(лістинг 7.13):

INSERT INTO transport (number, type, capacity, driver_id)

VALUES (:number, :type, :capacity, :driver_id);

Лістинг 7.13 – Додавання нового транспорту

Додавання нового маршруту (задача A10, 8)(лістинг 7.14):

INSERT INTO routes (route_id, distance)

VALUES (:route_id, :distance);

Лістинг 7.14 – Додавання нового маршруту

Додавання зупинок маршруту (A10, 9)(лістинг 7.15):

INSERT INTO stops (route_id, stop_name, stop_order)

VALUES (:route_id, :stop_name, :order);

Лістинг 7.15 – Додавання зупинок маршруту

Отримання статистики за транспортом (задача 3, 10, A11)(лістинг 7.16):

SELECT type, COUNT(*) AS total

FROM transport

GROUP BY type;

Лістинг 7.16 – Отримання статистики за транспортом

Запити для моніторингу та контролю

Для підтримки стабільності системи реалізовані запити, що дозволяють отримувати аналітичну інформацію та проводити аудит дій користувачів.

Перевірка завантаженості маршруту транспортом (задача 3, 10)(лістинг 7.17):

SELECT r.route_id, r.start_point, r.end_point, COUNT(t.transport_id) AS total_transport

FROM routes r

```
JOIN transport t ON r.route_id = t.route_id
```

```
GROUP BY r.route_id;
```

Лістинг 7.17 – Перевірка завантаженості маршруту транспортом

Таким чином, розроблені SQL-запити забезпечують повноцінну роботу інформаційної системи міського транспорту – від реєстрації користувачів і подання заявок до управління маршрутами, транспортом і персоналом.

У процесі виконання SQL-запитів автоматично спрацьовують тригери, описані в додатку Б:

при оновленні статусу заявки (UPDATE job_applications) спрацьовує тригер №1, який встановлює дату прийняття рішення (задача A4, U3);

при додаванні або зміні транспорту (INSERT, UPDATE transport) спрацьовує тригер №2, що перевіряє унікальність призначення водія (задачі A2, A3);

при схваленні заявки (UPDATE job_applications) спрацьовує тригер №3, який автоматично змінює роль користувача (задачі A4, A6, U2).

8 ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ

Розглянемо програмну реалізацію на прикладі завдань адміністратора та водія, які відповідають за керування транспортом і маршрутами в інформаційній системі міського транспорту.

8.1 Підключення до бази даних

Перед виконанням будь-яких операцій користувач повинен бути авторизованим у системі. Кожна дія виконується з урахуванням його унікального ідентифікатора (user_id) та ролі (role_id), що визначає рівень доступу.

Для підключення до бази даних у NestJS використовується TypeORM, який забезпечує автоматичне керування з'єднанням із PostgreSQL (лістинг 8.1):

```
import { TypeOrmModuleOptions } from '@nestjs/typeorm';

import { Role } from '../entity/role.entity';

import { User } from '../entity/user.entity';

import { Transport } from '../entity/transport.entity';

import { Route } from '../entity/route.entity';

import { Stop } from '../entity/stop.entity';

import { JobApplication } from '../entity/job-application.entity';
```



```

export const ormConfig: TypeOrmModuleOptions = {

  type: 'postgres',

  host: process.env.DB_HOST || 'localhost',

  port: +(process.env.DB_PORT || 5050),

  username: process.env.DB_USER || 'postgres',

  password: process.env.DB_PASS || 'stas2004',

  database: process.env.DB_NAME || 'Transports',

  synchronize: false,

  logging: true,

  entities: [User, Role, Transport, Route, Stop, JobApplication],

  logger: 'advanced-console',

};

```

Лістинг 8.1 – Реалізація підключення до бази даних

8.2 Призначення транспортного засобу

Після авторизації адміністратор може призначити транспортний засіб через спеціальну форму. На сервері ця дія реалізована функцією, наведеною у (лістингу 8.2).

```

@Post('assign')

@Roles('Водій', 'Адміністратор')

assign(@Body() body: { userId: number; transportId: number }){

  return this.transportService.assignTransport(body.userId, body.transportId);

}

```

Лістинг 8.2 – Реалізація призначення транспорту

```

async assignTransport(userId: number, transportId: number){

  const transport = await this.transportRepo.findOne({

    where: { transportId },

    relations: ['driver'],

  });
}

```

```

if (!transport) throw new NotFoundException('Транспорт не знайдено');

if (transport.driver){

    if (transport.driver.userId === userId) return transport;

    throw new BadRequestException(

        'Цей транспорт вже закріплено за іншим водієм'

    );

}

const user = await this.userRepo.findOne({ where: { userId } });

if (!user) throw new NotFoundException('Користувача не знайдено');

transport.driver = user;

await this.transportRepo.save(transport);

return { message: 'Транспорт закріплено' };

}

```

Лістинг 8.2

8.3 Динамічне формування HTML-сторінки (фрагмент шаблону)

```
{transport ? (
```

Назва: {transport.name}

Тип: {transport.type}

```
): (
```

До вас не закріплено транспорт

}}

Лістинг 8.3 – Відображення статистики доходу в інтерфейсі

Дані, отримані з бази даних, динамічно передаються у шаблон, де відображаються у вигляді таблиці або графіка. Завдяки розділенню шарів представлення та бізнес-логіки, оновлення даних не потребує зміни структури HTML-шаблонів.

Таким чином, реалізація інтерфейсу взаємодії з базою даних включає декілька основних компонентів. Спочатку відбувається підключення до СУБД PostgreSQL за допомогою TypeORM, що забезпечує зв'язок між серверною частиною застосунку та базою даних. Далі серверні функції (контролери) відповідають за обробку HTTP-запитів користувачів, перевірку автентифікації та виконання SQL-запитів.

Подібна архітектура забезпечує зручність розширення та супроводу системи, а також дозволяє швидко інтегрувати новий функціонал – наприклад, модулі аналітики, обліку транспорту, водіїв або маршрутів. Повний код реалізації зазначених компонентів наведено у додатку В.

9 БЕЗПЕКА ІНФОРМАЦІЙНОЇ СИСТЕМИ

Безпека інформаційної системи є критичним аспектом при розробці платформи для управління міським транспортом, оскільки вона забезпечує захист персональних даних користувачів, безпечну роботу з маршрутами, транспортом та обліковими записами водіїв і диспетчерів, а також запобігає несанкціонованому доступу. У проєкті безпека реалізована комплексно – як на рівні серверної частини, так і на рівні бази даних PostgreSQL.

9.1. Ролі користувачів та їхні привілеї

У системі визначено чітку ієрархію ролей, кожна з яких має власний набір прав на об'єкти бази даних і доступ до функціональних можливостей системи:

Адміністратор (admin) – має повний доступ до всіх даних і операцій, включаючи додавання, редагування, видалення та перегляд користувачів, транспорту, маршрутів, заявок водіїв.

Водій (driver) – має доступ до перегляду власного транспорту, маршруту.

Користувач (user) – має право переглядати інформацію транспортні засоби, зупинки, маршрути.

Інформація про права доступу для кожного типу користувачів користувачів наведені в таблиці 9.1.

Позначення: C – створення (Create), R – читання (Read), U – оновлення (Update), D – видалення (Delete).

Розмежування доступу реалізовано за допомогою SQL-полей PostgreSQL.

Для кожної категорії користувачів створюється відповідна роль (admin, driver, passenger), якій надаються лише необхідні права відповідно до її функцій. Такий підхід реалізує принцип мінімальних привілеїв, згідно з яким користувач отримує лише ті права, які необхідні для виконання його завдань.

Таблиця 9.1. Привілеї ролей на основні об'єкти БД:

Об'єкти БД

Ролі

Пасажир

Водій

Адміністратор

Таблиці

Users

R

RU

CRUD

Transport

R

RU

CRUD

Routes

R

RU

CRUD

Stops

R

RU

CRUD

Applications

-

C

CRUD

Roles

-
-

CRUD

Для кожної категорії користувачів створюється відповідна роль (admin, driver, passenger), якій надаються лише необхідні права відповідно до її функцій. Такий підхід реалізує принцип мінімальних привілеїв, згідно з яким користувач отримує лише ті права, які необхідні для виконання його завдань. Контроль доступу реалізовано на рівні застосунку за допомогою рольової моделі, JWT-автентифікації та middleware і guards. Обмеження доступу до функціональних можливостей системи здійснюється відповідно до ролі користувача.

Доступ до бази даних організовано з використанням окремого користувача PostgreSQL із обмеженими правами доступу. Застосунок підключається до бази даних не під суперкористувачем postgres, а під спеціально створеним обліковим записом, який має лише необхідні привілеї.

Користувачу бази даних надано такі права:

підключення до бази даних (CONNECT)

виконання операцій SELECT, INSERT, UPDATE, DELETE над таблицями

При цьому заборонено:

створення та видалення таблиць (CREATE, DROP)

зміна структури бази даних (ALTER)

адміністративні привілеї (SUPERUSER)

Такий підхід реалізує принцип мінімальних привілеїв на рівні бази даних і забезпечує додатковий рівень захисту, навіть у разі компрометації серверної частини застосунку.

Запит на створення користувача з обмеженим доступом (лістинг 9.1):

```
CREATE USER transport_app_user WITH PASSWORD 'strong_password';
```

```
GRANT CONNECT ON DATABASE "Transports" TO transport_app_user;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE
```

```
ON ALL TABLES IN SCHEMA public
```

```
TO transport_app_user;
```

Лістинг 9.1 – Запит на створення користувача з обмеженим доступом

9.2 Механізм зберігання паролів

У системі паролі користувачів ніколи не зберігаються у відкритому вигляді. Перед записом у таблицю `users` паролі хешуються на рівні серверної частини за допомогою бібліотеки `bcrypt`, яка інтегрована в середовище `NestJS`. Звернення до бази даних відбувається під користувачем `transport_app_user`, який має обмежені права доступу.

Перед збереженням пароль обробляється функцією (лістинг 9.3):

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Лістинг 9.3 – Хешування пароля перед збереженням

Після цього у базу даних записується вже готовий хеш (лістинг 9.4):

```
await this.userRepository.save({  
  
  name,  
  
  email,  
  
  password: hashedPassword,  
  
  role,  
  
});
```

Лістинг 9.4 – Збереження користувача з хешованим паролем

Таким чином, навіть у разі витоку даних із таблиці користувачів, реальні паролі залишаються захищеними. Перевірка пароля при вході здійснюється через метод (лістинг 9.5):

```
const isMatch = await bcrypt.compare(password, user.password);
```

Лістинг 9.5 – Перевірка чи вірний пароль ввів користувач

що не дає можливості отримати оригінальний пароль із хешу, навіть у випадку його перехоплення.

9.3 Алгоритм автентифікації та визначення ролі користувача

1 Введення даних: користувач вводить логін (`email`) і пароль на сторінці входу.

Перевірка: сервер отримує дані, знаходить користувача в базі даних і порівнює хеш пароля з введеним значенням через `bcrypt.compare()` (перевірка відбувається під користувачем `transport_app_user`, який має обмежені права доступу).

Визначення ролі: якщо автентифікація успішна, система читає поле `role` і формує JWT-токен із зазначенням ролі (`admin`, `driver`, `passenger`).

Авторизація: при кожному запиті токен перевіряється через `Guard` у `NestJS`, який визначає, чи має користувач право на виконання певної дії.

Приклад перевірки ролі у контролері (лістинг 9.6):

```
@UseGuards(JwtAuthGuard, RolesGuard)

@Roles('Адміністратор')

@Get('all-users')

async getAllUsers() {

  return this.userService.findAll();

}
```

Лістинг 9.6 – Контролер для перевірки ролі

Якщо користувач не має потрібної ролі, запит автоматично відхиляється з повідомленням про відмову в доступі (HTTP 403 Forbidden).

9.4 Додаткові механізми захисту

Для запобігання SQL-ін'єкціям у системі використовується ORM `TypeORM`, яка повністю ізолює користувацькі дані від прямого підставлення у SQL-запити. Кожен запит формується через параметри або методи репозиторіїв, наприклад (лістинг 9.7):

```
await this.transportRepository.find({

  where: { type: 'Bus', status: 'active' },

});
```

Лістинг 9.7 – Знаходження транспорту за певними критеріями

`TypeORM` автоматично обробляє вхідні параметри, що унеможлиблює SQL-ін'єкції навіть при введенні шкідливих символів.

Крім цього, у системі реалізовано:

Захист `CORS` – доступ до API дозволено лише з довірених доменів (лістинг 9.8).

```
app.enableCors({  
  
  origin: 'http://localhost:3000',  
  
  credentials: true,  
  
});
```

Лістинг 9.8 – Налаштування CORS

HTTP-only JWT cookies — токени аутентифікації зберігаються в захищених cookies, недоступних з JavaScript.

Шифрування конфіденційних даних через HTTPS.

Валідацію вхідних даних за допомогою class-validator, що запобігає некоректним запитам.

Таким чином, безпека системи «Міський транспорт» реалізована на всіх рівнях — від збереження паролів і керування ролями користувачів до обмеження запитів та контролю доступу в реальному часі. Особлива увага приділяється захисту облікових даних, що передбачає використання сучасних алгоритмів хешування та механізмів запобігання несанкціонованому доступу. Рольова модель доступу забезпечує чітке розмежування прав між різними категоріями користувачів, що мінімізує ризики зловживань та внутрішніх загроз.

10 ІНСТРУКЦІЯ КОРИСТУВАЧА

10.1 Інтерфейс пасажирів

При переході на ресурс в самому початку можна побачити сторінку з логіну (рис. 10.1). На даному моменті користувач може залогінитися або

zareestruvatisya.

Рисунок 10.1 – Сторінка логіну

Зверху можна побачити дві кнопки реєстрація або вхід (рис. 10.2), zareestruvatisya.

Рисунок 10.2 – Меню для реєстрації або логіну

Увівши свою пошту, пароль та натиснувши «Увійти», користувач побачить головну сторінку застосунка (рис. 10.3), яка відображає інформацію про пасажирів – а саме , ім'я, фамілія та роль.

Рисунок 10.3 – Сторінка профілю авторизованого користувача з навігаційним меню

Якщо користувач помилився при вводі пошти або паролю то він побачить повідомлення про помилку(рис. 10.4) «Неправильний email або пароль».

Рисунок 10.4 – Повідомлення про неправильний email або пароль при вході в особистий кабінет

Наступне вікно інтерфейсу пасажирів – це вікно «Транспорт»

(рис. 10.5). У даному вікні користувач може переглянути список усіх транспортів та за ким вони закріплені.

Рисунок 10.5 – Вікно відображення транспорту

На сторінці «Маршрути» (рис. 10.6) користувач має змогу переглянути всі маршрути, їх відстань та кількість зупинок і їх послідовність.

Рисунок 10.6 – Вікно маршрутів

У вікні «Подати заявку на роботу» (рис. 10.7, 10.8) користувач має можливість залишати заявку на роботу за обраною роллю що доступні. При переході на сторінку користувач одразу бачить форму, у якій потрібно обрати потрібну йому роль за якої він хоче отримати роботу. Після вибору ролі користувач натискає на кнопку подати заявку після чого він бачить вікно в якому написано роль на яку він подав заявку та її статус.

Рисунок 10.7 – Форма для створення заявки

Рисунок 10.8 – Вікно заявки користувача

10.2 Інтерфейс водія

Якщо користувач виявився водієм його переносить на окрему сторінку «Dashboard» (рис. 10.9). На ній він так само може побачити інформацію про себе: ім'я, фамілія, роль та в нього з'являється додаткова кнопка «Мій транспорт».

Рисунок 10.9 – Головна сторінка водія

Натиснувши кнопку «Мій транспорт» водій побачить детальну інформацію про свій транспорт «Транспорт» (рис. 10.10).

Рисунок 10.10 – Вікно з переліком об'єктів оренди орендодавця

10.3 Інтерфейс адміністратора.

На головній панелі адміністратора «Дашборд» (рис. 10.11) відображається інформація про адміністратора та з'являються дві додаткові кнопки «Користувачі» та «Заявки».

Рисунок 10.11 – Сторінка «Дашборд»

Вікно управління користувачами (рис. 10.12), дозволяє адміністратору

переглядати детальну інформацію по кожному користувачу, сортувати по різним параметрам та створювати, редагувати та видаляти користувачів.

Рисунок 10.12 – Вікно управління користувачами

Вікно «Заявки» (рис. 10.13), є вікном де адміністратор бачить всі заявки від користувачів з їх статусом та може підтвердити її або відхилити

Рисунок 10.13 – Сторінка «Заявки»

Вікно «Аналітики» (рис 10.14), є вікном де адміністратор переглядає статистику найпопулярніших маршрутів за кількістю призначених транспортів на маршрут.

Рисунок 10.14 – Сторінка «Аналітика»

ВИСНОВКИ

У результаті виконання курсової роботи розроблено інформаційну систему міського транспорту, яка реалізує комплексний підхід до управління перевізним процесом і взаємодії між адміністраторами, водіями та користувачами. Під час проєктування системи виконано повний цикл створення програмного продукту – від аналізу предметної області та побудови інформаційної моделі до розробки інтерфейсу користувача та реалізації механізмів безпеки.

Розроблена система має тришарову архітектуру, що включає:

клієнтський рівень, побудований на Next.js, який забезпечує інтуїтивно зрозумілий інтерфейс і швидку взаємодію користувача з даними.

серверний рівень на базі NestJS, що реалізує бізнес-логіку, авторизацію та управління запитами.

рівень даних із використанням PostgreSQL і TypeORM, який гарантує надійність і структурованість збереження інформації.

Особливу увагу приділено модульності системи, що дозволяє легко розширювати її функціонал. У межах проєкту реалізовано модулі авторизації, керування користувачами, транспортом, маршрутами, заявками та ролями.

Інформаційна система відзначається високим рівнем безпеки – використано JWT-аутентифікацію, хешування паролів, middleware і guards для перевірки прав доступу, а також захист від CORS-атак. Це гарантує безпечну роботу навіть у разі активної взаємодії великої кількості користувачів.

Система дозволяє автоматизувати основні процеси управління міським транспортом: адміністратори можуть керувати транспортом і персоналом, водії отримують інформацію про маршрути, а користувачі – зручний доступ до даних про перевезення. Також реалізовано механізм подання заявок на роботу, що надає можливість розширювати штат водіїв без залучення сторонніх систем.

Розроблена інформаційна система має не лише навчальну, а й практичну цінність. Її можна адаптувати для використання в реальних умовах – у муніципальних транспортних службах або приватних перевізників.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

NestJS – офіційна документація. – Режим доступу: <https://docs.nestjs.com>

PostgreSQL – офіційна документація. – Режим доступу: <https://www.postgresql.org/docs/>

TypeORM – офіційна документація. – Режим доступу: <https://typeorm.io/>

Next.js – офіційна документація. – Режим доступу: <https://nextjs.org/docs>

TailwindCSS – офіційна документація. – Режим доступу: <https://tailwindcss.com/docs>

Material UI – офіційна документація. – Режим доступу: <https://mui.com/>

ISO/IEC 9126 – Software Engineering – Product Quality.

Мартін Р. «Чиста архітектура. Мистецтво розробки програмного забезпечення». – К.: ВД «Фактор», 2021.

W3C Recommendations – HTML, CSS, DOM Standards.

Mozilla Developer Network (MDN) – Режим доступу: <https://developer.mozilla.org>

ДОДАТОК А
Запити на створення таблиць бази даних

```
CREATE TABLE roles (  
  
    roleId SERIAL PRIMARY KEY,  
  
    role VARCHAR(255) NOT NULL UNIQUE  
  
);
```

Лістинг А.1 – Створення таблиці role

```
CREATE TABLE users (  
  
    userId SERIAL PRIMARY KEY,  
  
    name VARCHAR,  
  
    surname VARCHAR,  
  
    age INTEGER,  
  
    email VARCHAR NOT NULL,  
  
    password VARCHAR NOT NULL,  
  
    "roleId" INTEGER,
```

```
CONSTRAINT pk_users PRIMARY KEY ("userId"),
```

```
CONSTRAINT uq_users_email
```

```
    UNIQUE (email),
```

```
CONSTRAINT fk_users_role
```

```
    FOREIGN KEY ("roleId")
```

```
    REFERENCES public.roles ("roleId")
```

```
    ON DELETE SET NULL,
```

```
CONSTRAINT chk_users_age
```

```
    CHECK (age IS NULL OR age BETWEEN 14 AND 120),
```

```
CONSTRAINT chk_users_email_format
```

```
    CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
```

```
);
```

Лістинг А.2 – Створення таблиці users

```
CREATE TABLE transport (
```

```
    transportId SERIAL PRIMARY KEY,
```

```
    type VARCHAR NOT NULL,
```

```
    "number" VARCHAR,
```

```
    capacity INTEGER NOT NULL,
```

transport_condition VARCHAR NOT NULL,

driver_id INTEGER,

CONSTRAINT pk_transport PRIMARY KEY ("transportId"),

CONSTRAINT fk_transport_driver

FOREIGN KEY (driver_id)

REFERENCES public.users ("userId")

ON DELETE SET NULL,

CONSTRAINT chk_transport_capacity

CHECK (capacity BETWEEN 1 AND 34),

);

Лістинг А.3 – Створення таблиці transport

CREATE TABLE routes (

routeId SERIAL PRIMARY KEY,

route VARCHAR(255) NOT NULL,

distance INT NOT NULL

CONSTRAINT chk_routes_distance

CHECK (distance >= 1 AND distance <= 30)

);

Лістинг А.4 – Створення таблиці routes

CREATE TABLE stops (

```
stopId SERIAL PRIMARY KEY,  
  
stop_name VARCHAR(255) NOT NULL,  
  
);
```

Лістинг А.5 – Створення таблиці stops

```
CREATE TABLE route_stops (  
  
id SERIAL PRIMARY KEY,  
  
routeId INT NOT NULL,  
  
stopId INT NOT NULL,  
  
stop_order INT NOT NULL,  
  
  
  
CONSTRAINT fk_rs_route  
  
    FOREIGN KEY (routeId)  
  
    REFERENCES routes(routeId)  
  
    ON DELETE CASCADE,  
  
  
  
CONSTRAINT fk_rs_stop  
  
    FOREIGN KEY (stopId)  
  
    REFERENCES stops(stopId)  
  
    ON DELETE CASCADE,  
  
  
  
CONSTRAINT uq_route_stop_order  
  
    UNIQUE (routeId, stop_order)  
  
);
```

Лістинг A6 – Створення проміжної таблиці між таблицями stops та route

```
CREATE TABLE IF NOT EXISTS job_applications (  
  
  id          SERIAL PRIMARY KEY,  
  
  status      VARCHAR NOT NULL DEFAULT 'pending',  
  
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),  
  
  decision_at TIMESTAMP,  
  
  user_id     INTEGER,  
  
  desired_role_id INTEGER,  
  
  
  CONSTRAINT fk_job_applications_user  
  
    FOREIGN KEY (user_id)  
  
    REFERENCES public.users (userId)  
  
    ON DELETE CASCADE,  
  
  
  CONSTRAINT fk_job_applications_role  
  
    FOREIGN KEY (desired_role_id)  
  
    REFERENCES public.roles (roleId)  
  
);
```

Лістинг A.6 – Створення таблиці job_applications

ДОДАТОК БЗапиту на створення тригерів та функції для вирішення задач IC

Функція призначена для встановлення дати прийняття рішення стосовно заявки на роботу (прийнята або відхилена). Відповідає задачі A4, U3 – розгляд заявок та перегляд статусу заявки.

```
CREATE OR REPLACE FUNCTION set_decision_date()  
  
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
IF NEW.status IN ('approved', 'rejected') AND OLD.status = 'pending' THEN
```

```
    NEW.decision_at := NOW();
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_set_decision_date
```

```
BEFORE UPDATE ON job_applications
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION set_decision_date();
```

Лістинг Б.1 – Функція встановлення дати прийняття рішення

Функція для перевірки щоб один водій мав лише один транспорт. Відповідає задачі A2, A3, V1 – облік транспорту та закріплення за водієм.

```
CREATE OR REPLACE FUNCTION check_driver_transport()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
IF NEW.driver_id IS NOT NULL THEN
```

```
    IF EXISTS (
```

```
        SELECT 1 FROM transport
```

```
        WHERE driver_id = NEW.driver_id
```

```
        AND transportId != NEW.transportId
```

```
    ) THEN
```

```
        RAISE EXCEPTION 'Цей водій вже має транспорт';
```

```
    END IF;
```



```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_driver_transport
```

```
BEFORE INSERT OR UPDATE ON transport
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION check_driver_transport();
```

Лістинг Б.2 - Функція для перевірки щоб один водій мав лише один транспорт

Функція для автоматичної активації водія. Відповідає задачі А4, А6, U2 – подання заявки, її схвалення та зміна ролі користувача.

```
CREATE OR REPLACE FUNCTION approve_driver_role()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
IF NEW.status = 'approved' THEN
```

```
    UPDATE users
```

```
    SET "roleId" = NEW.desired_role_id
```

```
    WHERE "userId" = NEW.user_id;
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_approve_driver_role
```

```
AFTER UPDATE ON job_applications
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION approve_driver_role();
```

Лістинг Б.3 – Функція для автоматичної активації водія

ДОДАТОК ВОсновні модулі NestJs

auth.service.ts

```
async login(user: any, res: any){

  const payload = {

    userId: user.userId,

    email: user.email,

    role: user.role,

  };

  console.log(payload);

  const token = this.jwtService.sign(payload);

  res.cookie('token', token, {

    httpOnly: true,

    secure: process.env.NODE_ENV === 'production',

    sameSite: 'lax',

    maxAge: 1000 * 60 * 60,

  });

  return {

    access_token: token,

    user: {

      userId: user.userId,

      email: user.email,

      role: user.role.roleName,

      name: user.name,

      surname: user.surname,
```

```
},  
  
};  
  
}
```

Лістинг В.1 – Функція логіну

transport.service.ts

```
async assignTransport(userId: number, transportId: number){  
  
    const transport = await this.transportRepo.findOne({  
  
        where: { transportId },  
  
        relations: ['driver'],  
  
    });  
  
    if (!transport) throw new NotFoundException('Транспорт не знайдено');
```

Лістинг В.2 – Функція призначення транспорту

applications.service.ts

```
async approve(id: number){  
  
    const app = await this.appRepo.findOne({  
  
        where: { id },  
  
        relations: ['user', 'desiredRole'],  
  
    });  
  
    console.log(app);  
  
    if (!app){  
  
        throw new NotFoundException('Заявку не знайдено');  
  
    }  
  
    app.status = 'approved';  
  
    app.decisionAt = new Date();  
  
    app.user.role = app.desiredRole;  
  
    await this.userRepo.save(app.user);
```

```
await this.appRepo.save(app);

return { message: 'Заявку схвалено' };

}
```

Лістинг В.3 – Функція прийняття заявки

ДОДАТОК ГОСНОВНІ КОМПОНЕНТИ ФРОНТЕНДУ

LoginForm.tsx

```
'use client';

import { useState } from 'react';

import { useRouter } from 'next/navigation';

import { TextField, Button, Card, CardContent } from '@mui/material';

import api from '@lib/api';

import { useAuth } from '@contexts/AuthContext';

export default function LoginPage() {

  const [email, setEmail] = useState("");

  const [password, setPassword] = useState("");

  const { login } = useAuth();

  const router = useRouter();

  const handleSubmit = async (e: React.FormEvent) => {

    e.preventDefault();

    try {

      const res = await api.post('/auth/login', { email, password });

      console.log(res);

      await login(res.data.access_token);

      router.push('/');

    } catch (err) {

      alert('Невірний email або пароль');
```

```
}  
};  
  
return(  

```

Вхід

```
<TextField  
  label='Email'  
  fullWidth  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
/>  
  
<TextField  
  label='Пароль'  
  type='password'  
  fullWidth  
  value={password}  
  onChange={(e) => setPassword(e.target.value)}  
/>
```

Увійти

```
);  
}
```

Лістинг Г.1 – Компонент форми логіну

DashboardPage

```
'use client';  
  
import { useAuth } from '@contexts/AuthContext';  
  
import { Button, Card, CardContent } from '@mui/material';  
  
export default function DashboardPage(){  
  
  const { user, logout } = useAuth();  
  
  if (!user){  
  
    return
```

Завантаження...

```
;  
  
}  
  
return(  
  

```

Вітаю, {user.name} {user.surname}

Вийти

```
);  
}
```

Лістинг Г.2 – Головна сторінка

middleware.ts

```
import { NextResponse } from 'next/server';  
  
import type { NextRequest } from 'next/server';  
  
export function middleware(req: NextRequest){  
  
  const token =  
  
    req.cookies.get('token')?.value || req.headers.get('Authorization');  
  
  const protectedRoutes = [  
  
    '/',  
  
    '/users',  
  
    '/applications',  
  
    '/transport',  
  
    '/routes',  
  
  ];  
  
  if (protectedRoutes.some((path) => req.nextUrl.pathname.startsWith(path))) {  
  
    if (!token) {
```

```
    return NextResponse.redirect(new URL('/login', req.url));  
  
  }  
  
}  
  
return NextResponse.next();  
  
}  
  
export const config = {  
  matcher: [  
  
    '/',  
  
    '/users/:path*',  
  
    '/applications/:path*',  
  
    '/transport/:path*',  
  
    '/routes/:path*',  
  
  ],  
  
};
```

Лістинг Г.3 – Функція для перевірки чи залогінений користувач

ДОДАТОК ДЕР-діаграма БД Інформаційної системи міського транспорту

Рисунок Д.1 - ER-діаграма бази даних Інформаційної системи міського транспорту

Plagiarised content by URLs: